mindlog · id

Cryptographic Whitepaper

mindlog id Engineering
milo@mindlog.today

May 2026 – version 1.0

Abstract

mindlog · id is an open-source digital identity and end-to-end encrypted messenger published under the AGPLv3 licence. This document specifies the cryptographic mechanisms shipped in the current release: the *Signal-style* pairwise channel (X3DH key agreement + Double Ratchet), the group channel (sender keys + ECDSA), the multi-device key vault (PBKDF2 + AES-256-GCM, optionally unlocked via WebAuthn PRF), the out-of-band identity verification ladder (safety numbers / QR), and the ephemerality and one-shot message extensions. All primitives are NIST-standard or IETF-standard; no custom cipher or curve is introduced. A canonical reference implementation (`src/ratchet.ts`) is shared verbatim between the web and Android clients, and is exercised by a frozen set of test vectors (`test/vectors/ratchet.json`) replayed by both runtimes to guarantee bit-for-bit interoperability.

1 Threat model and design goals

We assume an active network adversary with full control over the transport layer. We additionally assume that the server (operated by the publisher or self-hosted) is honest-but-curious: it is trusted to deliver messages in order on a best-effort basis but is not trusted with plaintext content, attachments, key material, or call media.

The design goals are:

- **Confidentiality and authenticity** of every direct and group message under standard cryptographic assumptions (ECDLP on NIST P-256, AES-GCM security, HMAC-SHA-256 PRF).
- **Forward secrecy:** compromise of long-term key material must not retroactively expose past messages.
- **Post-compromise security (self-healing):** after a single healthy round-trip following a state compromise, the channel returns to a secure state.
- **No mandatory directory:** no phone number or email is required to open a channel.
- **Auditability and portability:** a single reference algorithm spec, verbatim across runtimes, validated by shared test vectors.

2 Primitives

All primitives are NIST or IETF specified. We use the same set on web (WebCrypto, `crypto.subtle`) and on Android (`javax.crypto` + `java.security`).

- **Elliptic curve:** NIST P-256 (`secp256r1`) for both key agreement (ECDH) and signatures (ECDSA).

- **KDF:** HKDF-SHA-256 (RFC 5869) with explicit, domain-separated `info` labels.
- **Symmetric AEAD:** AES-256-GCM with a fresh 96-bit IV per message and a 128-bit authentication tag.
- **MAC / chain advance:** HMAC-SHA-256.
- **Password-based KDF:** PBKDF2-HMAC-SHA-256 with 600 000 iterations (OWASP 2023+ recommendation) for the vault unlock path.
- **ECDSA encoding:** fixed-length P1363 ($r\|s$, 64 bytes) for cross-runtime interoperability.

3 Pairwise channel

Each pair of identities runs an instance of the Signal-family **Double Ratchet** initialised by **X3DH**.

3.1 Identity and prekeys

Every account publishes:

- an identity public key IK_A (P-256),
- a medium-term signed prekey SPK_A (rotated),
- a pool of one-time prekeys $\{OPK_A^{(i)}\}$.

The server stores only the *public* bundle and consumes one-time prekeys atomically, gated per requesting contact, so that a single OPK is never handed out twice.

3.2 X3DH initial key agreement

To open a channel toward B, the initiator A generates an ephemeral EK_A and computes:

$$DH_1 = \mathrm{ECDH}(IK_A, SPK_B), \qquad DH_2 = \mathrm{ECDH}(EK_A, IK_B),$$
$$DH_3 = \mathrm{ECDH}(EK_A, SPK_B), \qquad DH_4 = \mathrm{ECDH}(EK_A, OPK_B^{(i)}) \text{ (if available).}$$

The initial root key is $RK_0 = \mathrm{HKDF}(DH_1 \| DH_2 \| DH_3 \| DH_4,\ \text{info} = \text{“mindlog/x3dh”})$.

3.3 Double Ratchet

State per channel: a 32-byte root key RK, a sending chain key CK_s, a receiving chain key CK_r, and a Diffie–Hellman ratchet pair.

Symmetric ratchet step. For each message sent, the chain key is advanced and a message key is derived:

$$mk = \mathrm{HMAC}(CK_n, \texttt{0x01}), \qquad CK_{n+1} = \mathrm{HMAC}(CK_n, \texttt{0x02}).$$

The message key mk is then split via HKDF into an AES-256 encryption key and a 12-byte IV. The ciphertext is $ct = \text{AES-GCM}_{mk}(\text{header}, \text{plaintext})$, binding the header (sender DH public, counters) as Associated Data.

DH ratchet step. On receipt of a message advertising a new sender DH public DH', the receiver computes $\mathrm{ECDH}(\cdot, DH')$, mixes it into RK, and resets both chain keys:

$$(RK', CK_r') = \mathrm{HKDF}(RK \| \mathrm{ECDH}(sk, DH'),\ \text{info} = \text{“mindlog/dr”}).$$

This produces *forward secrecy* (past message keys are unrecoverable after a chain advance) and *post-compromise security* (one healthy round-trip restores secrecy after a state compromise).

3.4 Wire format

A message is packaged as

$$\texttt{ciphertext} = \texttt{b64url}(\text{header}) \| \texttt{"."} \| \texttt{b64}(ct),$$

with a header version flag (`v=1`, `v=2`) for forward compatibility. No new database column is required: the server is oblivious to the channel state and stores opaque blobs only.

4 Group channel

Group conversations use **sender keys**, mirroring the Signal group construction, with the simplification that the group admin is the creator and membership transitions are authenticated by the admin's pairwise channel.

Each group member M holds a *sender state*: a 32-byte symmetric chain key CK^M and a P-256 ECDSA signing key pair $(sk^M_{\text{sig}}, pk^M_{\text{sig}})$.

4.1 Sender Key Distribution Message (SKDM)

On joining or on rotation, M ships its initial CK^M and pk^M_{sig} to every other member *over their pairwise Double Ratchet channel*. The SKDM therefore inherits the confidentiality, authenticity, forward secrecy, and post-compromise security properties of the pairwise channel; the server only sees a regular E2EE message tagged with the control prefix `skd`.

4.2 Encrypt and decrypt

To send to the group, M advances its chain:

$$mk = \text{HKDF}(CK^M_n, \text{info} = \text{“mindlog/sk/mk”}), \quad CK^M_{n+1} = \text{HKDF}(CK^M_n, \text{info} = \text{“mindlog/sk/ck”}).$$

The payload is $ct = \text{AES-GCM}_{mk}(\text{header}, \text{plaintext})$, and the entire (header, ct) is signed: $\sigma = \text{ECDSA}_{sk^M_{\text{sig}}}(\text{header} \| ct)$, encoded in fixed-length P1363 (64 bytes). Receivers verify σ against the sender's registered pk^M_{sig}, then decrypt.

4.3 Rotation

On any membership change that reduces the visibility set (member removal), each remaining member regenerates a fresh CK^M and re-distributes it via SKDM to every other current member. This is the standard sender-key forward-secrecy boundary on member removal.

5 Multi-device key vault

The user's identity private key must travel between devices without ever being exposed to the server in cleartext. We store it as an *encrypted vault* with two unwrap paths.

Passphrase path. The vault key is derived as

$$K_{\text{vault}} = \text{PBKDF2-HMAC-SHA256}(\text{passphrase},\ \text{salt}_{\text{vault}},\ 600\,000),$$

producing 32 bytes; the identity key is then sealed with $\text{AES-GCM}_{K_{\text{vault}}}$ under a fresh 12-byte IV. The server stores the resulting ciphertext, salt, and IV only; the passphrase never leaves the device.

Passkey (WebAuthn PRF) path. Where the platform exposes the WebAuthn `prf` extension, the vault is unlocked by a 32-byte PRF output bound to the user's passkey, yielding a phishing- and replay-resistant unlock with no shared secret to remember.

The same envelope format is used on web and Android, so a vault written on one platform unlocks bit-identically on the other.

6 Identity verification

To resist a server-side man-in-the-middle attack on the initial X3DH bundle, each pair derives a *safety number*:

$$\mathrm{SN}_{A,B} = \text{SHA-256}(\min(IK_A, IK_B) \| \max(IK_A, IK_B)),$$

encoded as a stable 60-digit string and rendered as a QR code. Two users that scan or compare each other's safety number obtain an out-of-band authentication of their channel that does not depend on the server's honesty.

7 Ephemerality, attachments, view-once

Disappearing messages. The header may carry an explicit TTL τ (seconds). Both endpoints schedule a deterministic deletion at $t_{\text{recv}} + \tau$; the ciphertext on the server is purged on the same deadline.

View-once messages. A view-once flag instructs the recipient to delete plaintext, ciphertext, and header from local storage as soon as the message is decrypted and rendered, and to acknowledge a server- side *burn* for the corresponding row. The message key is also zeroized after a single decryption.

Attachments and voice. Files are encrypted in chunks with a per-attachment AES-256-GCM key freshly derived from the per-message key material. The server stores opaque ciphertext blobs and never sees a filename, MIME type, or plaintext byte.

8 Directoryless contact (invite links)

A user A may mint a single-use invite token t and share the URL `https://id.mindlog.today/i/<t>`. The token is bound to A on the server but does not reveal A's identity attributes; upon acceptance, both sides install a mutual *contact* relation and immediately run the X3DH exchange described above. The token is destroyed after first use. No phone number, no email, and no directory lookup are involved.

9 Test vectors and cross-runtime parity

The reference algorithm lives in `src/ratchet.ts` and is exercised by the frozen test vectors `test/vectors/ratchet.j`. The Android runtime ships unit tests that replay the same vectors against its `javax.crypto` / `java.security` stack (`GroupSenderTest`, `RatchetTest`). Any algorithmic divergence between web and Android fails CI, which is what makes the multi-client *universality* of the cryptographic core a checked property rather than a documentation claim.

10 What is not done

We do not claim post-quantum security in this release. We do not claim metadata privacy: the server learns who talks to whom and when, although not what is said. There is no sealed-sender construction, no mixnet routing, and no PIR-based contact discovery. These are tracked items on the roadmap.

References

- Marlinspike, M. and Perrin, T. *The X3DH Key Agreement Protocol.* Signal Foundation, 2016. `https://signal.org/docs/specifications/x3dh/`

- Perrin, T. and Marlinspike, M. *The Double Ratchet Algorithm.* Signal Foundation, 2016. `https://signal.org/docs/specifications/doubleratchet/`
- Signal Foundation. *Sender Keys for Group Messaging.*
- W3C. *Web Authentication, Level 3 – PRF Extension.* `https://www.w3.org/TR/webauthn-3/`